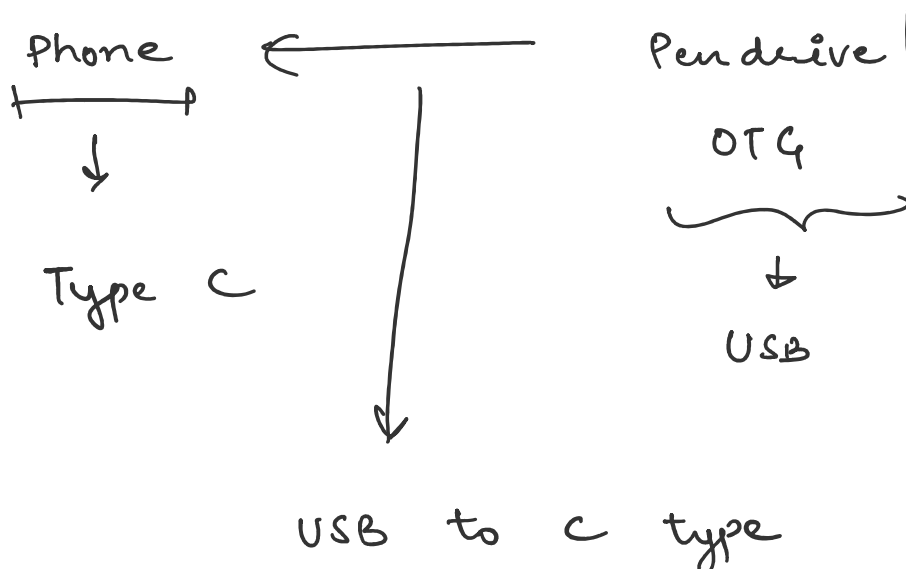


Adapter Design Pattern

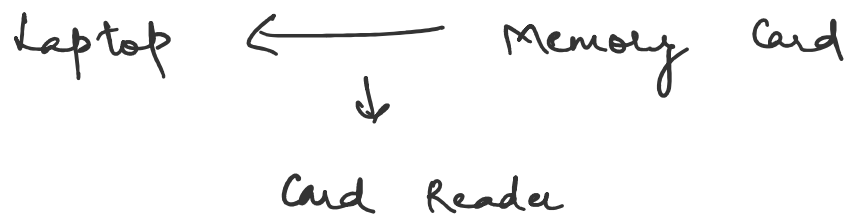
US, UK, UAE → universal Adapter → charge our devices



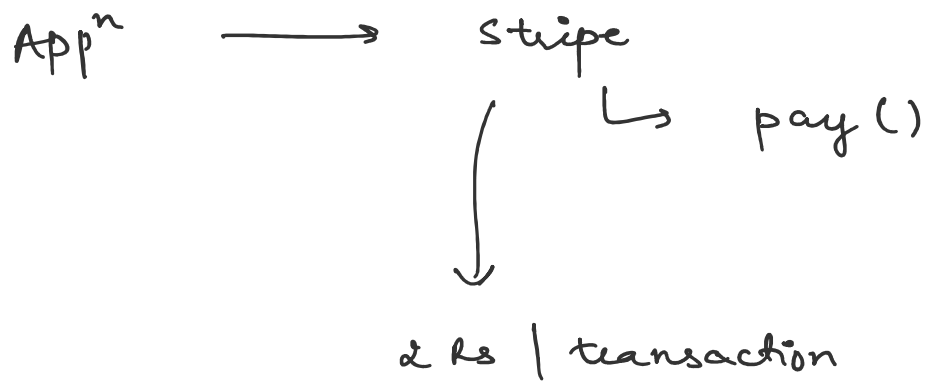
Allows two incompatible power supply and charger pin to work



Card Reader



Payment Example



New Method



Third party sdk



Paypal → makePayment()

Razorpay → processPayment()

Appⁿ

pay()

use
Adapter



"
Bridge
"

{ makePayment() =
processPayment() =



Incompatible

call API



Client → Target Interface → expected by
the client



Adapter

→ providing
support



Adaptee

→ Third party
existing classes

Another example -

e-commerce Appⁿ → Send Notification()



sending notification to customers



Initially only emails



Add support for WhatsApp and SMS



WhatsApp

SDK



Meta



send WhatsApp Msg()



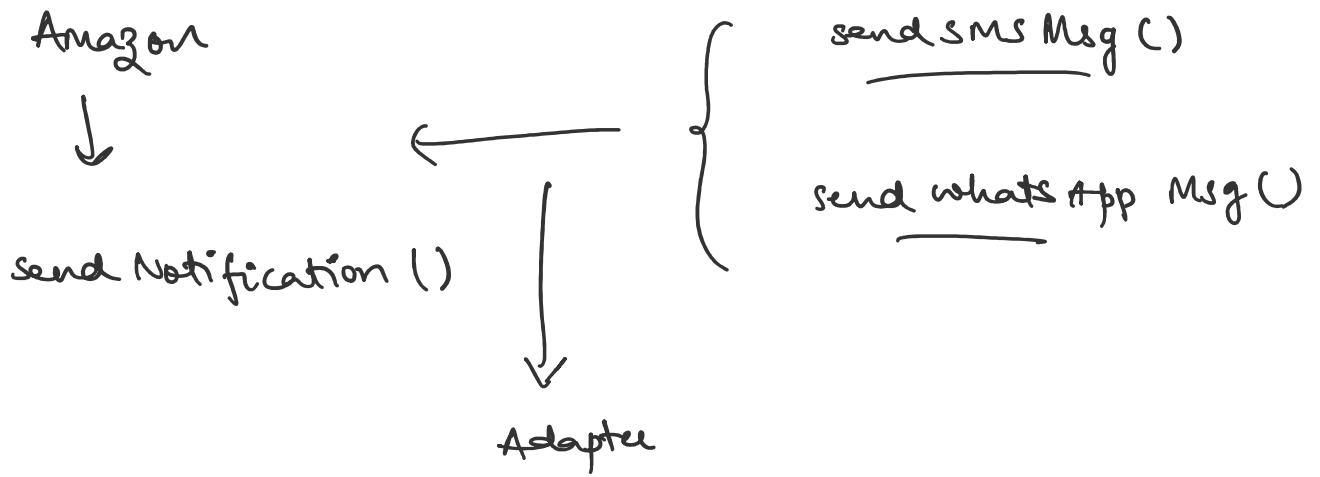
SMS SDK



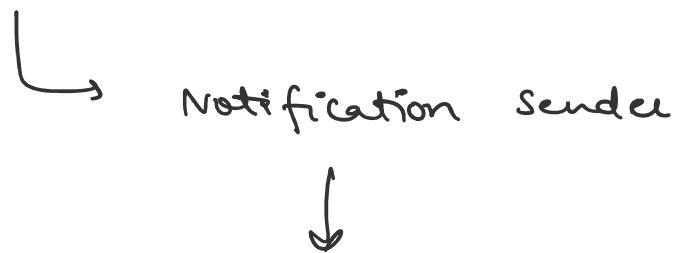
Apple



send SMS Msg()



Target Interface



sendNotification (sender, receiver)

Existing Email Sender

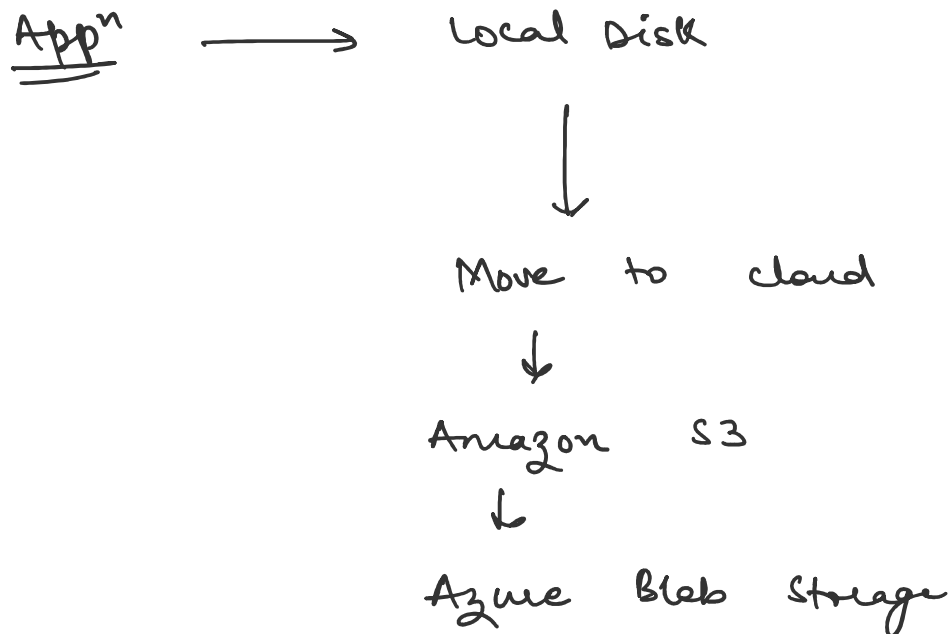


send Email (sender , receiver)



Already Supported

Another eg- cloud storage system



Local Disk
↳ upload (string fileName)

Amazon S3
↳ putObject (string bucket ,
string key ,
string fileName)

Azure Blob Storage

└─ upload Blob (string container ,
string blobName ,
string fileName)

Notes:

Adapter Design Pattern

The Adapter Pattern is a **structural design pattern** that allows two incompatible interfaces to work together.

Sometimes we already have a class that provides the functionality we need, but its interface is different from what our application expects. Instead of modifying the existing class (which may be third-party or legacy code), we create an **Adapter** that converts one interface into another.

Think of it as a translator between two systems.

The Adapter allows the client to use the existing class without knowing that an adaptation is happening behind the scenes.

The Adapter acts like a **bridge** between them and helps one system work with another without changing their code.

Structure

The Adapter Pattern consists of four participants.

1. Client

The code that wants to use a service. The client depends **only on the Target interface** and is completely unaware that an adapter is translating requests behind the scenes.

2. Target

The interface expected by the client. Every implementation that the client works with must implement this interface.

Example:

```
public interface PaymentGateway {  
    void pay(double amount);  
}
```

3. Adaptee

The existing class that already provides the required functionality but exposes a different interface.

Usually this is

- Third-party SDK
- Legacy code
- External API
- Vendor library

The client cannot use it directly because its interface doesn't match the Target.

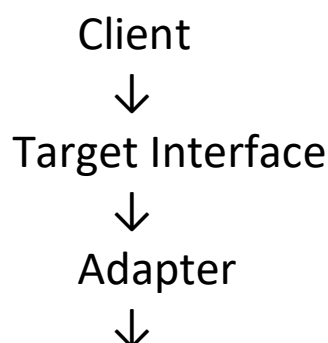
Example:

```
public class PayPalSDK {  
    public void makePayment(double amount){}  
}
```

4. Adapter

The adapter is the bridge between the **Client** and the **Adaptee**.
It

- **Implements the Target interface**
- **Contains an instance of the Adaptee**
- Converts the client's request into the format expected by the **Adaptee**
- Delegates the actual work to the **Adaptee**



Adaptee

When should you use Adapter?

Use the Adapter Pattern when:

- **You want to reuse an existing class, but its interface is incompatible with your application.**
- **You are integrating a third-party SDK or vendor library.**
- **You are working with legacy code that cannot be modified.**
- **Different systems expose different APIs, but your application expects a common interface.**
- **You want to support new providers without changing existing business logic.**

A simple rule to remember:

If the functionality already exists, but the interface doesn't match, use an Adapter.

Real-life Analogies

1. Phone Charger (AC to DC Conversion)

Problem: When you plug your phone into a wall socket, the electricity coming from the socket is **AC (Alternating Current)** and usually around **240V**.

But your phone **cannot handle AC**, and it doesn't need that much voltage — it typically needs **DC (Direct Current)** at **5V**.

If you plug the phone directly into the socket without a charger, it will likely **burn or get damaged**, because the **interface (power type and voltage)** doesn't match.

Solution: The **charger** acts as an **adapter** between the **socket (240V AC)** and your **phone (5V DC)**.

- It **converts the interface** from AC to DC.
- It **reduces the voltage** from 240V to 5V.
- It ensures your phone can **safely and correctly use** the electricity.

In programming, this is exactly what the Adapter Pattern does — it allows a **client expecting one interface** to work with an **existing class with a different interface**, without modifying either.

Wall Socket (Adaptee)

|

Charger (Adapter)

|

Phone (Client)

2. OTG / Pen-drive → Phone (USB → Type-C)

Problem: Your **pen drive** uses a **USB-A** connector, which is designed for computers. Your **smartphone**, however, uses a **USB-C (Type-C)** or

micro-USB port.

These two hardware interfaces are **incompatible** — you can't just plug a pen drive into your phone.

Solution: An **OTG (On-The-Go) adapter** solves this problem. It:

- **Converts the USB-A interface** to USB-C/micro-USB.
- Makes it possible for the phone to **read data** from the pen drive.
- Works **without modifying** the pen drive or the phone.

Think of your **app** (the phone) trying to read data from a **legacy service** (pen drive). The OTG adapter is like a **software adapter class** that converts the legacy service's interface into something your app understands.

Pen Drive

|

OTG Adapter

|

Phone

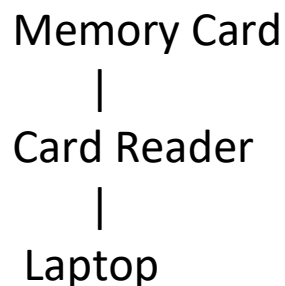
3. Card Reader (Memory Card + Laptop)

Problem: A **memory card** cannot be plugged directly into most laptops unless the laptop has a specific slot. The **interface** between the memory card and the laptop is **incompatible**.

Solution: A **card reader** is used here:

- You **insert the memory card** into the reader.
- You **plug the reader** into the laptop via USB.
- The card reader **adapts the memory card interface** into a standard USB interface, which the laptop can understand.

This is like writing an adapter that allows a new **data format or class** to work with your system **without changing your system's input expectations**.



Payment Processing System

Suppose your application originally supports only **Stripe**. Later, customers ask for **PayPal** and **Razorpay**.

The problem is each provider exposes a different **API**. Your application expects only one interface.

Instead of changing your business logic every time, we create adapters.

Step 1. Target Interface

```
public interface PaymentGateway {  
    void pay(double amount);  
}
```

Step 2. Existing Stripe Implementation

```
public class StripeGateway implements PaymentGateway {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Stripe Payment: $" + amount);  
    }  
}
```

Stripe already follows our interface. No adapter is required.

Step 3. Third-party PayPal SDK

Suppose PayPal provides this SDK. We cannot modify it.

```
public class PayPalSDK {  
    public void makePayment(double amount) {  
        System.out.println("PayPal Payment: $" + amount);  
    }  
}
```

Notice the method name differs.

Step 4. Third-party Razorpay SDK

```
public class RazorpaySDK {  
    public void processTransaction(double amount) {  
        System.out.println("Razorpay Payment: ₹" + amount);  
    }  
}
```

Different method again.

Third-party SDKs and legacy systems should generally never be modified.

Reasons:

- Source code may not be available.
- Vendor updates would overwrite your changes.
- Other applications may also depend on the SDK.
- Modifying external libraries makes maintenance difficult.

Instead of changing the SDK, we write a small Adapter that translates our application's interface into the SDK's interface.

Step 5. Adapter for PayPal

Notice that **StripeGateway** already implements the interface expected by our application. Since it already follows the **Target** interface, the client can use it directly.

However, **PayPal** exposes **makePayment()** while **Razorpay** exposes **processTransaction()**. Their interfaces don't match **PaymentGateway**, so adapters are required.

```

public class PayPalAdapter implements PaymentGateway {
    private final PayPalSDK payPalSDK;

    public PayPalAdapter(PayPalSDK payPalSDK) {
        this.payPalSDK = payPalSDK;
    }

    @Override
    public void pay(double amount) {
        payPalSDK.makePayment(amount);
    }
}

```

The Adapter Pattern follows one of the most important OO principles: **Composition over Inheritance**. Instead of extending the existing class, the adapter **contains** the existing object.

Example:

```

public class PayPal {
    public void makePayment(double amount) {
        System.out.println("Processing payment of $" + amount + "
through PayPay.");
    }
}

```

The adapter simply delegates work.

```

pay(amount)
  ↓
payPalSDK.makePayment(amount)

```


Benefits

- Loose coupling
- Better flexibility
- Easier testing
- Can wrap any implementation

Step 6. Adapter for Razorpay

```
public class RazorpayAdapter implements PaymentGateway {
    private final RazorpaySDK razorpaySDK;

    public RazorpayAdapter(RazorpaySDK razorpaySDK) {
        this.razorpaySDK = razorpaySDK;
    }

    @Override
    public void pay(double amount) {
        razorpaySDK.processTransaction(amount);
    }
}
```

Step 7. Client

```
public class Main {
    public static void main(String[] args) {
        PaymentGateway stripe = new StripeGateway();
        stripe.pay(1000); // Output : Stripe Payment: $1000.0

        PaymentGateway payPal = new PayPalAdapter(new
        PayPalSDK());
        payPal.pay(2000); // Output : PayPal Payment: $2000.0
    }
}
```

```
PaymentGateway razorpay = new RazorpayAdapter(new
RazorpaySDK());
razorpay.pay(3000); // Output : Razorpay Payment: ₹3000.0
}
```

Notification System (Email, SMS & WhatsApp)

You built an e-commerce application that sends notifications to customers.

Initially, your application only supported **Email** notifications. Later, the business wanted to support **SMS** and **WhatsApp** notifications.

The problem is that each third-party provider exposes a different interface (API), while your application expects a single common interface.

Instead of modifying your business logic whenever a new provider is added, you can use the **Adapter Pattern**.

1. Create the Target Interface

This is the interface your application understands.

```
public interface NotificationSender {
    void sendNotification(String sender, String receiver);
}
```

```
}
```

2. Create Existing Email Sender (Already Supported)

Email already follows the expected interface, so no adapter is needed.

```
public class EmailSender implements NotificationSender {  
    @Override  
    public void sendNotification(String sender, String receiver) {  
        System.out.println("Sending email notification from " +  
sender + " to " + receiver);  
    }  
}
```

3. Create Third-Party SMS SDK (Adaptee)

Imagine this class comes from SMS's SDK. We cannot modify it.

```
public class SMSSDK {  
    public void sendSMS(int phoneNumber, String message) {  
        System.out.println("Sending SMS to " + phoneNumber + "  
with message : " + message);  
    }  
}
```

Notice that the method is **sendSMS()**, not **send()**.

4. Create Third-Party WhatsApp SDK (Adaptee)

```
public class WhatsAppSDK {
```

```

    public void sendWhatsAppMessage(int mobileNum, String
text) {
        System.out.println("Sending WhatsApp message to " +
mobileNum + " : " + text);
    }
}

```

Again, the interface is different.

5. Create SMS Adapter

The adapter converts the application's expected interface into Twilio's interface.

```

public class SMSAdapter implements NotificationSender {
    private final SMSSDK smssdk;

    public SMSAdapter(SMSSDK smssdk) {
        this.smssdk = smssdk;
    }

    @Override
    public void sendNotification(String sender, String receiver) {
        // sender -> phoneNumber
        // receiver -> message
        smssdk.sendSMS(Integer.parseInt(sender), receiver);
    }
}

```

6. Create WhatsApp Adapter

```

public class WhatsAppAdapter implements NotificationSender {

```

```

private final WhatsAppSDK whatsAppSDK;

public WhatsAppAdapter(WhatsAppSDK whatsAppSDK) {
    this.whatsAppSDK = whatsAppSDK;
}

@Override
public void sendNotification(String sender, String receiver) {
    // mapping values
    // sender -> mobile number -> parsing it to int to match the
datatype
    // receiver -> text

    whatsAppSDK.sendWhatsAppMessage(Integer.parseInt(sender),
    "Message from " + receiver);
}
}

```

7. Client Code

```

public class Main {
    public static void main(String[] args) {
        NotificationSender emailSender = new EmailSender();
        emailSender.sendNotification("sender@example.com",
        "receiver@example.com"); // Output : Sending email notification
        from sender@example.com to receiver@example.com

        NotificationSender smsSender = new SMSAdapter(new
        SMSSDK());
        smsSender.sendNotification("1234567890", "Hello, this is a
        test SMS!"); // Output : Sending SMS notification from
        1234567890 to Hello, this is a test SMS!

        NotificationSender whatsAppSender = new
        WhatsAppAdapter(new WhatsAppSDK());
    }
}

```

```
    whatsAppSender.sendNotification("987654321", "Hello, this  
is a test WhatsApp message!"); // Output : Sending WhatsApp  
message to 987654321 : Message from Hello, this is a test  
WhatsApp message!  
    }  
}
```

Cloud Storage System

Imagine you're building a document management platform. Initially, all documents are stored on the local server.

Later, enterprise customers request support for cloud storage. Different customers want different providers:

- Amazon S3
- Azure Blob Storage
- Google Cloud Storage

Each provider exposes its own SDK and method names.

Amazon S3

putObject(bucket,key,file)

Azure Blob

uploadBlob(container,blob,file)

Google Cloud

uploadObject(bucket,file)

Your application shouldn't know these implementation details. Instead, it should simply call : **storage.upload(fileName);**

Without Adapter

```
if(provider == AMAZON)
    putObject()

else if(provider == AZURE)
    uploadBlob()

else if(provider == GCP)
    uploadObject()
```

Every new cloud provider requires changing your application. This violates the Open/Closed Principle.

1. Create Target Interface

```
public interface StorageService {
    void upload(String fileName);
}
```

2. Existing Local Storage

```
public class LocalStorage implements StorageService {
    @Override
    public void upload(String fileName) {
```

```
        System.out.println("Uploading " + fileName + " to Local  
Storage");  
    }  
}
```

3. Amazon S3 SDK (Adaptee)

```
public class AmazonS3SDK {  
    public void putObject(String bucketName, String objectKey,  
String fileName) {  
        System.out.println("Uploading " + fileName + " to Amazon  
S3 bucket : " + bucketName + ", using object key : " + objectKey);  
    }  
}
```

4. Azure Blob SDK (Adaptee)

```
public class AzureBlobSDK {  
    public void uploadBlob(String containerName, String  
blobName, String fileName) {  
        System.out.println("Uploading " + fileName + " to Azure Blob  
Container : " + containerName + ", inside blob : " + blobName);  
    }  
}
```

5. Google Cloud SDK (Adaptee)

```
public class GoogleCloudSDK {  
    public void uploadObject(String bucket, String objectName,  
String fileName) {
```



```
        System.out.println("Uploading " + fileName + " to Google  
Cloud Storage bucket : " + bucket + ", using object name : " +  
objectName);  
    }  
}
```

6. Amazon Adapter

```
public class AmazonAdapter implements StorageService {  
    private final AmazonS3SDK amazonS3SDK;  
  
    public AmazonAdapter(AmazonS3SDK amazonS3SDK) {  
        this.amazonS3SDK = amazonS3SDK;  
    }  
  
    @Override  
    public void upload(String fileName) {  
        String objectKey = "invoices/2026/" + fileName;  
        amazonS3SDK.putObject("documents", objectKey,  
fileName);  
    }  
}
```

7. Azure Adapter

```
public class AzureAdapter implements StorageService {  
    private final AzureBlobSDK azureBlobSDK;  
  
    public AzureAdapter(AzureBlobSDK azureBlobSDK) {  
        this.azureBlobSDK = azureBlobSDK;  
    }  
}
```

```

@Override
public void upload(String fileName) {
    String blobName = "employee-documents/" + fileName;
    azureBlobSDK.uploadBlob("company-files", blobName,
fileName);
}
}

```

8. Google Cloud Adapter

```

public class GoogleCloudAdapter implements StorageService {
    private final GoogleCloudSDK googleCloudSDK;

    public GoogleCloudAdapter(GoogleCloudSDK
googleCloudSDK) {
        this.googleCloudSDK = googleCloudSDK;
    }

    @Override
    public void upload(String fileName) {
        String objectName = "reports/2026/" + fileName;
        googleCloudSDK.uploadObject("company-docs",
objectName, fileName);
    }
}

```

9. Client Code

```

public class Main {
    public static void main(String[] args) {
        StorageService local = new LocalStorage();
        local.upload("resume.pdf"); // Output : Uploading resume.pdf
to Local Storage

```

```
StorageService amazon = new AmazonAdapter(new  
AmazonS3SDK());  
amazon.upload("photo.jpg"); // Output : Uploading photo.jpg  
to Amazon S3 bucket : documents, using object key :  
invoices/2026/photo.jpg
```

```
StorageService azure = new AzureAdapter(new  
AzureBlobSDK());  
azure.upload("invoice.pdf"); // Output : Uploading invoice.pdf  
to Azure Blob Container : company-files, inside blob : employee-  
documents/invoice.pdf
```

```
StorageService google = new GoogleCloudAdapter(new  
GoogleCloudSDK());  
google.upload("report.docx"); // Output : Uploading  
report.docx to Google Cloud Storage bucket : company-docs,  
using object key : reports/2026/report.docx  
}  
}
```

Advantages

- **Interface Compatibility:** Enables systems with mismatched interfaces to work together.
- **Code Reusability:** Use existing classes without modifying them.
- **Flexibility:** Add new adapters for new use cases (e.g., new currencies) without rewriting existing logic.
- **Easy to Test and Maintain:** Adapter logic is isolated; business logic remains untouched.

Where do you use it in real systems?

As an experienced engineer, you'll see scenarios like:

- **Legacy code integration:** New system needs to work with an old API that has a different interface and old system's APIs cannot be changed.
- **Third-party libraries:** The library provides one interface, but your app expects another.

Examples include:

- Stripe
- Razorpay
- PayPal
- Twilio
- Firebase
- AWS SDK
- Azure SDK
- Google Cloud SDK

Each SDK exposes different APIs. Adapters provide a common interface to the rest of the application.

- **Database Migration :** Suppose your application initially uses MySQL but later supports PostgreSQL or MongoDB.

Each database driver has different APIs. Adapters allow the application to work with a common repository interface.

- **Cloud Provider Integration** : Supporting multiple cloud providers without changing business logic.
- **Payment Gateway Integration** : Every payment provider exposes different APIs. The application simply calls

```
paymentGateway.pay(amount);
```

The adapter translates this into the provider-specific SDK call.